

Partie 3 : Des Transformers aux Large Language Models

Bassem Ben Hamed

ENETCOM – Université de Sfax

Février 2026

Section 3.1 — L'émergence des LLMs

- 1 L'évolution de la famille GPT
- 2 Écosystème LLM open source
- 3 Lois de mise à l'échelle et capacités émergentes
- 4 LLM multimodaux
- 5 Pipeline de pré-formation
- 6 Prédiction du jeton suivant
- 7 Stratégies de décodage
- 8 Fenêtre contextuelle et extensions

Section 3.2 — Prompt Engineering & Function Calling

- 9 Principes fondamentaux du Prompt Engineering
- 10 Zero-shot et Few-shot prompting
- 11 Raisonnement en chaîne de pensée
- 12 Prompting avancé (ToT, GoT)
- 13 Sorties structurées (mode JSON)
- 14 Notions de base sur les appels de fonctions
- 15 Appel de fonction avancée
- 16 Travaux pratiques : Ateliers 4 & 5

À la fin de cette session, vous serez capable de :

- ➊ **Tracer** l'évolution de GPT-1 à GPT-4 et comprendre les étapes clés.
- ➋ **Naviguer** l'écosystème open source LLM (LLaMA, Mistral, Phi, etc.).
- ➌ **Expliquer** les lois d'échelle et les capacités émergentes dans les grands modèles.
- ➍ **Comprendre** les pipelines de pré-entraînement et la génération autorégressive.
- ➎ **Appliquer** les stratégies de décodage (température, top-k, top-p) efficacement.
- ➏ **Maîtriser** les techniques de Prompt Engineering : Zero-shot, Few-shot, CoT.
- ➐ **Mettre en uvre** des workflows de Function Calling avec sorties structurées.

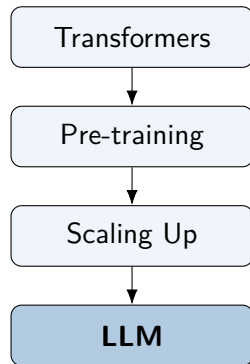
Section 3.1

L'émergence des Large Language Models

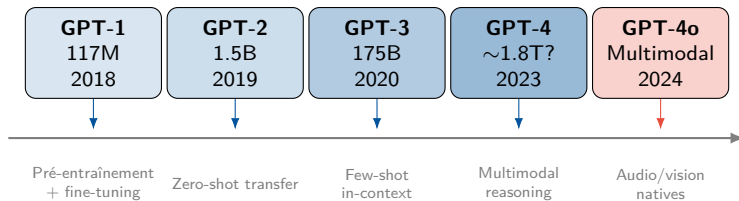
Scaling des Transformers, pré-entraînement & génération

Des transformateurs aux LLM

- Transformers (2017) a fourni l'architecture.
- Aperçu clé : paramètres, données et calcul **augmenter**.
- Trois changements de paradigme :
 - Pré-entraîner sur des données massives non étiquetées
 - Fine-tuning ou prompting pour les tâches en aval
 - Apprentissage en contexte sans mises à jour de poids
- Résultat : des modèles qui affichent de surprenants **capacités émergentes**.

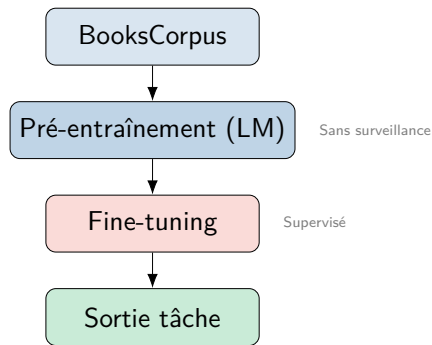


L'évolution de la famille GPT



GPT-1 : Pré-formation non supervisée + mise au point (2018)

- **Architecture** : transformateur 12 couches uniquement pour décodeur (117 M de paramètres).
- **Étape 1** : Pré-formation non supervisée sur BooksCorpus.
- **Étape 2** : Supervision de la mise au point des tâches en aval.
- **Innovation clé** : transfert d'apprentissage pour la PNL.
- A montré que la pré-formation générative améliore les tâches discriminantes.



Aperçu clé

Un seul modèle pré-entraîné peut être adapté à de nombreuses tâches avec très peu de changements d'architecture.

GPT-2 : scaling et Zero-shot learning (2019)

- **Échelle** : 1,5 B paramètres ($10\times$ GPT-1).
- **Données de formation** : WebText (40 Go de texte Web organisé).
- **Découverte clé** : Le modèle effectue les tâches *sans* de réglage fin.
- Zero-shot : demander au modèle une description de la tâche.
- Les modèles linguistiques sont des apprenants multitâches non supervisés.

Exemple Zero-shot

Prompt:

TL : Cest un marqueur qui annonce un résumé court.

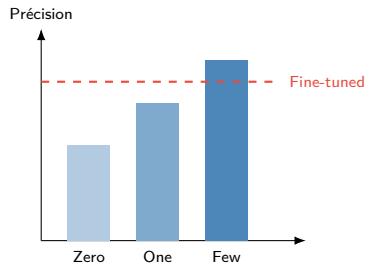
Résultat: Le modèle génère un résumé sans avoir été entraîné spécifiquement au résumé.

Controverse

OpenAI a d'abord retenu le modèle complet en invoquant des risques d'usage malveillant (génération de fake news).

GPT-3 : Few-shot In-Context Learning (2020)

- **Échelle** : 175 B de paramètres, 300 B de jetons de formation.
- **Percée** : Apprentissage en contexte (ICL).
- Aucune mise à jour des poids n'est nécessaire au moment de l'inférence.
- Trois modes : Zero-shot, One-shot, Few-shot.
- Les performances s'améliorent avec plus d'exemples dans le prompt.

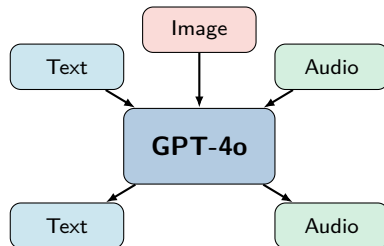


GPT-4 (mars 2023) :

- Multimodal : saisies texte + image.
- Réussite à l'examen du barreau (90e percentile).
- Fenêtre contextuelle de 128 Ko (GPT-4 Turbo).
- Architecture mixte d'experts (MoE) (rumeur).

GPT-4o (mai 2024) :

- Multimodal natif : texte + image + audio.
- Conversations vocales en temps réel.
- 2× plus rapide, 50 % moins cher que GPT-4 Turbo.



Écosystème LLM Open Source

Famille	Créateur	Tailles	Principales fonctionnalités
LLaMA 1/2/3	Meta	7B–405B	Open weights, strong baselines
Mistral / Mixtral	Mistral AI	7B–8×22B	MoE, efficient, multilingual
Phi-1/2/3/4	Microsoft	1.3B–14B	Small but powerful (SLMs)
Gemma 1/2	Google	2B–27B	Lightweight, research-friendly
Qwen 1/2	Alibaba	0.5B–72B	Multilingual, strong on Chinese
DeepSeek	DeepSeek	7B–236B	Code-specialized, MoE

Pourquoi l'Open Source est important

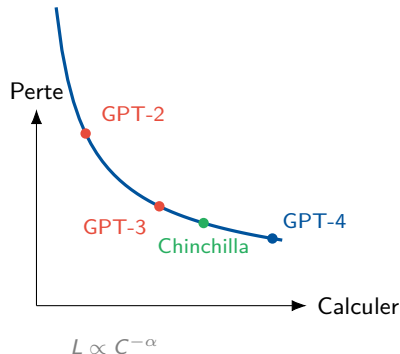
- Transparence et reproductibilité
- Mise au point sur des données propriétaires
- Déploiement sur site

Considérations

- Restrictions de licence (utilisation commerciale)
- Les modèles plus petits $\neq$ toujours pires
- Soutien communautaire ou soutien d'entreprise

Lois de mise à l'échelle : calcul, données, paramètres

- **Kaplan et al. (2020)** : la perte suit la loi de puissance avec la taille du modèle, les données et le calcul.
- $L(N) \propto N^{-\alpha}$ (la perte diminue de manière prévisible).
- **Chinchilla (Hoffmann 2022)** : le ratio optimal est de ~ 20 jetons par paramètre.
- La plupart des modèles avant Chinchilla étaient *sous-entraînés*.

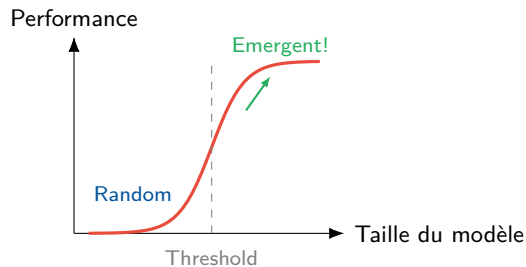


Aperçu Chinchilla

Un modèle 70B entraîné sur 1.4T tokens surpasse un modèle 280B entraîné sur 300B tokens.

Capacités émergentes à grande échelle

- Capacités qui apparaissent **soudainement** à une certaine échelle.
- Non présent dans les modèles plus petits, alors 'allumez'.
- Exemples observés :
 - Raisonnement en chaîne de pensée
 - Arithmétique en plusieurs étapes
 - Génération de code
 - Traduction multilingue
- Débat : sagit-il dun artefact de mesure ou dune véritable transition de phase ?



Modèles Vision-Langage :

- GPT-4V/4o : compréhension + génération d'images
- Claude Vision : analyse de documents, graphiques
- LLaVA : réglage des instructions visuelles open source
- Gemini : multimodal natif de A à Z

Audio et parole :

- Whisper : une reconnaissance vocale robuste
- GPT-4o : conversations vocales en temps réel
- Modèles unifiés traitant toutes les modalités

Applications :

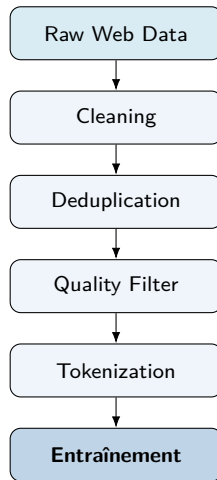
- Compréhension et sous-titrage des images
- Analyse de documents/PDF
- Compréhension vidéo
- Interprétation des figures scientifiques
- Outils d'accessibilité

Modèles d'architecture

- Encodeur vision + LLM (LLaVA)
- Multimodal natif (Gemini, GPT-4o)
- Fusion basée sur un adaptateur

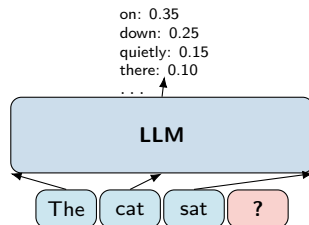
Pipeline de pré-formation

- **Apprentissage auto-supervisé** : aucune étiquette humaine n'est nécessaire.
- Objectif : prédire le prochain jeton en fonction de tous les jetons précédents.
- Nécessite **ensembles de données massifs** :
 - Common Crawl (~petabytes de site Web)
 - La pile (800 Go organisés)
 - RedPajama, RefinedWeb, FineWeb
 - Livres, Wikipédia, Code (GitHub)
- Étapes critiques du pipeline :
 - Déduplication (exacte + quasi-duplication)
 - Filtrage de qualité (perplexité, heuristique)
 - Suppression des contenus toxiques/nuisibles
 - Équilibrage de domaine



Prédiction du prochain jeton

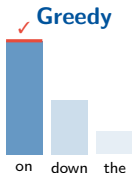
- **Génération autorégressive** : prédisez un jeton à la fois.
- $P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1})$
- Perte d'entraînement : **entropie croisée** entre le prochain jeton prévu et réel.
- $\mathcal{L} = -\sum_t \log P(x_t \mid x_{<t})$
- **Perplexité** : $\text{PPL} = \exp(\mathcal{L})$
 - Perplexité moindre = meilleur modèle.
 - Intuition : 'à quel point le modèle est-il surpris ?'



Stratégies de décodage (1/2) : Greedy contre Sampling

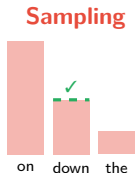
Décodage gourmand

- Choisissez toujours le jeton de probabilité la plus élevée.
- $x_t = \arg \max P(x \mid x_{<t})$
- Rapide mais répétitif et ennuyeux.
- Aucune diversité dans les sorties.



Échantillonnage aléatoire

- Échantillon de la distribution de probabilité complète.
- Plus diversifié mais peut être incohérent.
- Des jetons à faible probabilité peuvent être sélectionnés.
- Nécessite des mécanismes de contrôle.



Stratégies de décodage (2/2) : Température, Top-k, Top-p

Température τ

- $P'(x) = \frac{\exp(z_x/\tau)}{\sum \exp(z_i/\tau)}$
- $\tau \rightarrow 0$: gourmand
- $\tau = 1$: inchangé
- $\tau > 1$: plus aléatoire

Échantillonnage Top-k

- Ne conservez que k jetons de probabilité la plus élevée.
- Redistribuer la masse de probabilité.
- Coupure fixe quelle que soit la forme de la distribution.

Top-p (noyau)

- Gardez le plus petit ensemble où $\sum P \geq p$.
- S'adapte à la forme de la distribution.
- Plus flexible que top-k.
- Typique : $p = 0.9$

$\tau = 0.2$ (focused) $\tau = 1.0$ (normal) $\tau = 2.0$ (creative)



Pénalités de répétition et de fréquence

- **Problème** : les modèles autorégressifs ont tendance à se répéter.
- **Pénalité de répétition** : réduit la probabilité de jetons déjà générés.
- **Pénalité de fréquence** : pénalise les jetons proportionnellement à leur fréquence.
- **Pénalité de présence** : pénalise les jetons qui sont apparus.

Paramètre	Effet
temperature	Randomness
top_k	Hard token cutoff
top_p	Adaptive cutoff
rep. penalty	Reduce repeats
freq. penalty	Penalize frequent
pres. penalty	Penalize any seen

Fenêtre contextuelle et extensions

- **Fenêtre contextuelle** : nombre maximum de jetons que le modèle peut traiter à la fois.
- L'auto-attention a une complexité en séquence de $O(n^2)$.
- Évolution:
 - GPT-3 : jetons 2K/4K
 - GPT-4 : 8K/32K/128K
 - Claude : 200 000 jetons
 - Gemini : 1 million+ de jetons

Techniques d'extension :

- **Mise à l'échelle du RoPE** : interpoler ou extrapoler des plongements positionnels.
- **Alibi** : Attention aux biais linéaires (pas de plongements positionnels).
- **Fenêtre coulissante** : Attention locale (Mistral).
- **Attention distribuée** : répartir le calcul sur plusieurs appareils.

Compromis

Contexte plus long → plus de mémoire et une inférence plus lente, mais meilleure récupération "aiguille dans une botte de foin".

Paysage LLM : une comparaison

Modèle	Paramètres	Contexte	Ouvrir?	Modalités
GPT-4o	~1.8T (MoE)	128K	✗	Text, Image, Audio
Claude 3.5	Unknown	200K	✗	Text, Image
Gemini 1.5 Pro	Unknown	1M	✗	Text, Image, Audio, Video
LLaMA 3.1 405B	405B	128K	✓	Text
Mistral Large	Unknown	128K	✗	Text
Mixtral 8×22B	141B (39B active)	64K	✓	Text
Phi-3 Medium	14B	128K	✓	Text, Image
Qwen2 72B	72B	128K	✓	Text

Le paysage évolue rapidement — consultez les classements (LMSYS Chatbot Arena, Open LLM Leaderboard) pour les derniers classements.

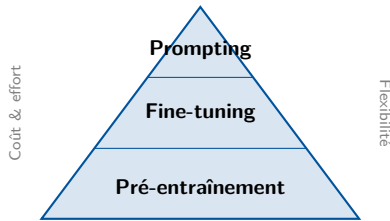
Section 3.2

Prompt Engineering, In-Context Learning & Function Calling

Piloter le comportement des LLMs sans entraînement

Pourquoi le Prompt Engineering ?

- Les LLM sont à **usage général** — les prompts les spécialisent.
- Aucune mise à jour de poids, aucune donnée d'entraînement nécessaire.
- Même modèle, prompts différents → comportements différents.
- Le 'langage de programmation' des LLM.



Comparaison des coûts

- Prompt Engineering : minutes, \$0
- Mise au point : heures/jours, \$100–\$10K+
- Pré-formation : semaines, \$1M–\$100M+

Composants du prompt

- 1 **Prompt système** : rôle, personnalité, contraintes.
- 2 **Contexte** : Informations générales.
- 3 **Instructions** : Que faire.
- 4 **Exemples** : (optionnel) démonstrations Few-shot.
- 5 **Entrée** : la requête / les données réelles.
- 6 **Format de sortie** : Structure attendue.

Exemple

Système: Vous êtes un développeur Python senior. Soyez concis.

Contexte: Nous utilisons FastAPI et PostgreSQL.

Instruction: Analysez le code suivant pour détecter les failles de sécurité.

Entrée: [extrait de code]

Format: Retournez une liste JSON des problèmes avec sévérité et correctif.

Zero-shot Prompting

- Aucun exemple fourni — juste l'instruction.
- S'appuie sur les connaissances pré-entraînées du modèle.
- Fonctionne bien pour les tâches courantes et bien définies.
- La performance dépend de la clarté du prompt.

Quand utiliser

- Tâches simples et sans ambiguïté
- Formats standards (traduction, résumé)
- Lorsque les exemples ne sont pas disponibles

Exemple

Classez le sentiment de l'avis suivant : positif, négatif ou neutre.

Avis : "Le repas était correct mais le service était catastrophique."

Sentiment :

Sortie du modèle

Négatif

Few-shot Prompting

- Fournissez 2 à 5 exemples d'entrées-sorties dans le prompt.
- Le modèle apprend le pattern à partir de démonstrations.
- Aucune mise à jour des poids — purement en contexte.
- L'ordre et la qualité des exemples comptent !

Meilleures pratiques

- Utiliser des exemples diversifiés et représentatifs
- Garder un format cohérent
- Inclure les cas extrêmes si possible

Exemple Few-shot

Classez le sentiment :

"Super produit !" → Positif

"Pire achat de ma vie" → Négatif

"C'est correct, sans plus" → Neutre

"La batterie est excellente
mais l'écran se fissure vite"

→

Sortie du modèle

Mixte (ou Neutre)

Raisonnement en chaîne de pensée (Wei et al. 2022)

- Inviter le modèle à **réfléchir étape par étape**.
- Améliore considérablement les tâches de raisonnement.
- **Few-shot CoT** : fournir des exemples de raisonnement.
- **Zero-shot CoT** : ajouter simplement `Réfléchissons étape par étape.`

Pourquoi ça marche

- Divise les problèmes complexes en sous-étapes
- Les jetons intermédiaires servent de `notes`
- Réduit la propagation des erreurs

Sans CoT

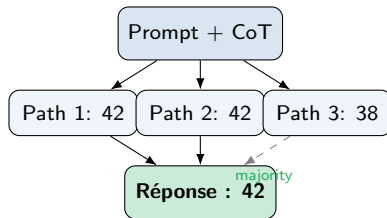
Q: Roger a 5 balles de tennis. Il achète 2 boîtes de 3. Combien ?
A: 11 ✗

Avec CoT

Q: Roger a 5 balles de tennis. Il achète 2 boîtes de 3. Combien ?
A: Roger part de 5 balles.
2 boîtes de 3 = 6 balles.
5 + 6 = 11 balles.
La réponse est 11. ✓

Auto-cohérence : plusieurs chemins de raisonnement

- Échantillonnez les chaînes de raisonnement **multiple** CoT (par exemple, 5 à 10).
- Prenez le **vote majoritaire** sur la réponse finale.
- Plus robuste qu'un simple CoT.
- Utilise une température plus élevée pour la diversité.
- Compromis : plus d'appels API = coût plus élevé.



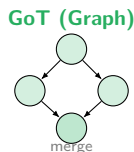
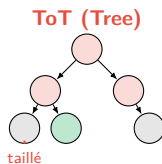
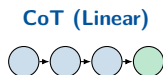
Prompting avancé : Tree-of-Thoughts et Graph-of-Thoughts

Arbre de pensées (ToT)

- Explorez plusieurs branches de raisonnement.
- Évaluez et élaguez les mauvais chemins.
- Utilisez les stratégies de recherche BFS ou DFS.
- LLM évalue chaque étape de la pensée.
- Bon pour : la planification, les puzzles, les tâches créatives.

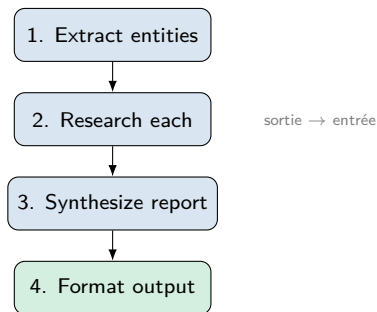
Graphique des pensées (GoT)

- Généralisation du ToT : permettre la fusion des chemins.
- Les pensées forment un graphique orienté, pas seulement un arbre.
- Peut combiner des solutions partielles.
- Bon pour : tri, optimisation, problèmes multi-contraintes.



Prompt chaining et décomposition

- Diviser les tâches complexes en **sous-prompts séquentiels**.
- La sortie de l'étape n alimente l'étape $n+1$.
- Avantages:
 - Chaque étape est plus simple et plus fiable
 - Plus facile à déboguer et à itérer
 - Peut mélanger des modèles/stratégies par étape
- Base pour des **workflows d'agents** (Partie 7).



Sorties structurées : mode JSON

- Forcer le LLM à produire un **JSON valide**.
- Critique pour utilisation programmatique des résultats du LLM.
- Méthodes :
 - Mode JSON au niveau API (OpenAI, Anthropic)
 - Génération contrainte par un schéma
 - Prompts contraints + validation
- **Pydantic** pour la validation de sortie en Python.

Exemple de sortie JSON

```
# Prompt:
"Extract entities as JSON:
'Apple CEO Tim Cook announced
new products in Cupertino.'"

# Output:
{
  "entities": [
    {"text": "Apple", "type": "ORG"},
    {"text": "Tim Cook", "type": "PER"},
    {"text": "Cupertino", "type": "LOC"}
  ]
}
```

Prompts système et définition des rôles

- **Prompt système** : instructions persistantes qui façonnent le comportement du modèle.
- Définit le **personnage**, les contraintes et le format de sortie.
- Séparé des messages utilisateur dans les appels API.
- La plupart des fournisseurs prennent en charge : les rôles système/utilisateur/assistant.

Exemple de prompt système efficace

You are a medical coding assistant specialized in ICD-10.

Rules:

- Only use official ICD-10 codes
- Always include code + description
- If unsure, say "uncertain"
- Never provide medical advice

Format de sortie : JSON avec les champs code, description, confidence.

Note de sécurité

Les prompts système peuvent être extraits via prompt injection. N'y mettez jamais de secrets.

À faire ✓

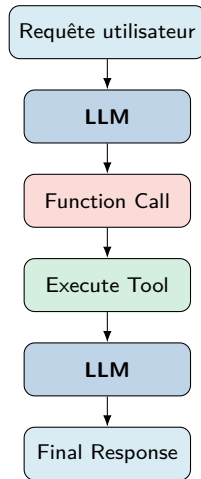
- Soyez précis et sans ambiguïté
- Utilisez des délimiteurs (" ", --, balises XML)
- Spécifier explicitement le format de sortie
- Fournir des exemples pour des tâches complexes
- Utilisez des instructions étape par étape
- Définir des contraintes (longueur, style, ton)

À ne pas faire

- Des instructions vagues (n faites quelque chose de bien z)
- Des exigences contradictoires
- Prompts trop longs avec un contexte non pertinent
- En supposant que le modèle n connaît z votre intention
- Ignorer la validation de sortie
- Mettre des secrets dans les prompts

Appel de fonction : connecter les LLM aux outils

- Les LLM peuvent é appeler é des fonctions/outils externes.
- Le modèle génère **appel de fonction** structuré (nom + arguments).
- L'application exécute la fonction et renvoie le résultat.
- Le modèle intègre le résultat pour générer la réponse finale.
- Pont clé de **génération de texte à action**.



Appel de fonction : définition d'outil (schéma JSON)

- Les outils sont définis en utilisant **Schéma JSON**.
- Précisez : nom, description, paramètres.
- La description est essentielle : elle indique au modèle *quand* d'utiliser l'outil.
- Les paramètres définissent la structure d'entrée attendue.

Définition de l'outil

```
{
  "name": "get_weather",
  "description": "Get current
    weather for a location",
  "parameters": {
    "type": "object",
    "properties": {
      "location": {
        "type": "string",
        "description": "City name"
      },
      "unit": {
        "type": "string",
        "enum": ["celsius",
          "fahrenheit"]
      }
    },
    "required": ["location"]
  }
}
```

Appel de fonction : exemple d'API OpenAI

Code Python

```
from openai import OpenAI
client = OpenAI()

tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get weather for a city",
        "parameters": {
            "type": "object",
            "properties": {
                "location": {"type": "string"},
                "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
            },
            "required": ["location"]
        }
    }
}]

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "What's the weather in Paris?"}],
    tools=tools,
    tool_choice="auto"    # let the model decide
)
```

Appel de fonction : exemple d'API anthropique

Code Python

```
import anthropic
client = anthropic.Anthropic()

tools = [{
    "name": "get_weather",
    "description": "Get weather for a location",
    "input_schema": {
        "type": "object",
        "properties": {
            "location": {"type": "string", "description": "City name"},
            "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
        },
        "required": ["location"]
    }
}]

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    tools=tools,
    messages=[{"role": "user", "content": "Weather in Tunis?"}]
)
# Check response for tool_use content blocks
```

Appel de fonction parallèle

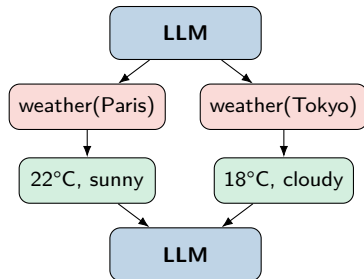
- Le modèle peut demander **plusieurs outils** en une seule réponse.
- Exécutez tout en parallèle pour plus de rapidité.
- Renvoyez tous les résultats ensemble.
- Le modèle synthétise la réponse combinée.

Exemple

Utilisateur : "Compare la météo à Paris et Tokyo."

Le modèle appelle :

```
get_weather("Paris")  
get_weather("Tokyo")  
simultanément.
```



Appel de fonction : gestion des erreurs & validation

Problèmes courants

- Le modèle hallucine des fonctions inexistantes
- Valeurs de paramètres invalides
- Échecs d'exécution de l'outil (API en panne, timeout)
- Le modèle interprète mal les résultats de l'outil

Meilleures pratiques

- Valider les noms de fonctions par rapport aux outils disponibles
- Valider les paramètres avec JSON Schema / Pydantic
- Renvoyer des messages d'erreur explicites au modèle
- Implémenter une logique de retry avec backoff exponentiel

Stratégie de repli

```
try:
    result = execute_tool(
        name=tool_call.name,
        args=tool_call.arguments
    )
except ToolNotFoundError:
    result = "Error: Unknown tool"
except ValidationError as e:
    result = f"Invalid args: {e}"
except TimeoutError:
    result = "Tool timed out"

# Return error to model
messages.append({
    "role": "tool",
    "content": result
})
```

Appel de fonction entre fournisseurs

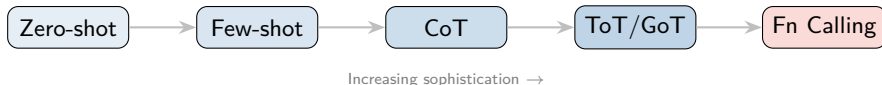
Fonctionnalité	OpenAI	Anthropic	Open Source
Définition d'outil	tools	tools	variable
Format du schéma	JSON Schema	input_schema	JSON Schema
Appels parallèles	✓	✓	selon le modèle
Forcer l'outil	tool_choice	tool_choice	limité
Streaming	✓	✓	variable
Mode JSON	response_format	prompt système	basé sur une grammaire

Clé à retenir

Les concepts sont identiques entre fournisseurs — seule la syntaxe API change. Le Function Calling converge vers un pattern commun.

Résumé : comparaison des techniques de prompting

Technique	Idéal pour	Complexité	Coût
Zero-shot	Simple, standard tasks	Low	Low
Few-shot	Custom formats, patterns	Medium	Low
CoT	Reasoning, math, logic	Medium	Medium
Self-Consistency	High-stakes reasoning	High	High
ToT / GoT	Planning, optimization	Very High	Very High
Prompt Chaining	Multi-step workflows	High	Medium
Function Calling	External tool integration	Medium	Medium



Points clés à retenir

- ➊ **L'échelle compte** : l'augmentation des paramètres, des données et des calculs entraîne des améliorations prévisibles (lois de mise à l'échelle).
- ➋ **Capacités émergentes** apparaissent à certains seuils d'échelle — raisonnement, génération de code, transfert multilingue.
- ➌ **Pré-formation** sur des données massives avec prédiction du prochain jeton crée de puissants modèles de base.
- ➍ **Stratégies de décodage** (température, top-k, top-p) contrôle le compromis créativité-cohérence.
- ➎ **Prompt Engineering** est le moyen le plus rapide de piloter le comportement du LLM — Zero-shot, Few-shot, CoT.
- ➏ **Appel de fonction** comble le fossé entre la génération de texte et les actions du monde réel.
- ➐ Le paysage LLM est **en évolution rapide** — les modèles fermés et open source progressent rapidement.

Atelier 4 : Interagir avec les modèles de base

- Charger et comparer des modèles open source (Llama 3, Phi-3, Mistral)
- Générez du texte avec différentes stratégies de décodage
- Analyser la température et l'impact de l'échantillonnage
- Visualisez les probabilités des jetons
- Comparez les tokenizers entre les familles de modèles
- Tester les capacités Zero-shot et Few-shot

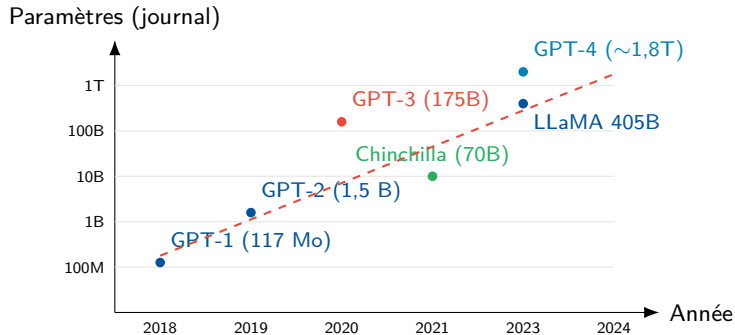
Atelier 5 : Prompt Engineering — Function Calling

- Implémenter Zero-shot, Few-shot et CoT
- Comparer les techniques de prompting entre les modèles
- Créer des workflows d'appel de fonctions
- Implémenter une extraction de sortie structurée
- Évaluer l'efficacité des prompts

Outils: Python, Hugging Face Transformers, OpenAI API, Anthropic API

- Radford et al. (2018) – Améliorer la compréhension du langage grâce à la pré-formation générative (GPT-1)
- Brown et al. (2020) – Les modèles linguistiques sont des apprenants peu nombreux (GPT-3)
- Kaplan et al. (2020) – Lois de mise à l'échelle pour les modèles de langage neuronal
- Hoffmann et al. (2022) - Formation LLM Compute-Optimal (Chinchilla)
- Wei et al. (2022) – Chain-of-Thought Prompting
- Yao et al. (2023) – Arbre des pensées
- Touvron et al. (2023) – LLaMA : LM de fondation ouverts et efficaces
- Jiang et al. (2023) – Mistral 7B
- Liu et al. (2023) – Réglage des instructions visuelles (LLaVA)

Annexe : Le nombre de paramètres LLM au fil du temps



Annexe : Comparaison des tokenizers

Tokeniseur	Algorithme	Taille du vocabulaire	Utilisé par
GPT-2 tokenizer	BPE	50,257	GPT-2
cl100k_base	BPE	100,256	GPT-3.5/4
o200k_base	BPE	200,019	GPT-4o
SentencePiece	Unigram/BPE	32K–128K	LLaMA, Mistral
WordPiece	WordPiece	30,522	BERT

Principaux compromis

- Vocabulaire plus grand → séquences plus courtes mais matrice d'intégration plus grande.
- Vocabulaire plus petit → séquences plus longues mais meilleure généralisation aux mots rares.
- La prise en charge multilingue nécessite des vocabulaires plus étendus.

Techniques

- **Zero-shot** : instruction directe uniquement
- **Few-shot** : 2 à 5 exemples
- **CoT** : “Réfléchissez étape par étape”
- **Auto-cohérence** : échantillons multiples + vote
- **ToT** : Explorez les branches du raisonnement
- **Chaînage** : sous-tâches séquentielles

Phrases magiques

- “Réfléchissons étape par étape”
- “Expliquez votre raisonnement”
- “Respirez profondément et ...”
- “Vous êtes un expert en ...”
- “Sortie uniquement du JSON valide”
- “En cas de doute, dites que je ne sais pas”